

Sentinel Desktop: A Fully Local, Evidence-Grounded Interactive Application Security Assistant

Anonymous Authors

Department of Computer Science and Engineering

Institution/University

City, Country

Email: author@institution.edu

Abstract—Web application security testing requires combining domain expertise, structured analysis, and reproducible verification. Recent advancements in Large Language Models (LLMs) have enabled AI-assisted security tools to support reconnaissance, subtask planning, and vulnerability analysis. However, several recent LLM-assisted systems rely on third-party cloud-hosted AI APIs—introducing data confidentiality risks—while suffering from technical hallucinations, CLI-only tooling boundaries, and unverified vulnerability reporting. This paper presents Sentinel Desktop, a fully local interactive application security assistant operating strictly over host local loopback (127.0.0.1). Sentinel Desktop integrates Chrome DevTools Protocol (CDP) browser instrumentation, automated passive security rule evaluation, non-destructive inert canary reflection probing, multi-source scanner ingestion (SARIF v2.1.0 and OWASP ZAP), deterministic replay comparison, and local LLM remediation advising. By confining network traffic strictly to local loopback environments, Sentinel Desktop reduces external telemetry exposure while grounding AI reasoning in cryptographically traceable session evidence. We evaluate the framework across 130 physical trials on a controlled local fixture server. Under the tested conditions, the replay comparison engine correctly classified 30/30 baseline vs. re-test trials ($N = 30$, mean latency 1.12 ms, maximum P_{95} latency 2.20 ms); the inert canary engine achieved 100.0% binary reflection detection ($N = 50$, mean latency 0.26 ms, maximum P_{95} latency 0.47 ms), with exact context classification at 70.0% overall—specifically `HTML_BODY` (15/15), `SCRIPT_STRING` (10/10), and `NOT_FOUND` (10/10)—identifying a sequential DOM scanning limitation where `ATTRIBUTE` reflections (15/15) were categorized under `HTML_BODY`; the passive rule engine correctly evaluated `CSP_MISSING_CHECK` across 30 response streams ($N = 30$, mean latency 0.36 ms, maximum P_{95} latency 0.82 ms); and external scanner ingestion achieved 100.0% mapping accuracy to OWASP 2021 categories for the tested report files ($N = 20$, mean latency 0.10 ms, maximum P_{95} latency 0.10 ms). The local LLM experiments were not executed during this evaluation because Ollama was not installed or running on the evaluation host. This work establishes an empirical, reproducible foundation for local AI-assisted application security inspection.

Index Terms—Web Application Security, Large Language Models, Local AI, Browser Automation, Vulnerability Verification, Replay Testing, Evidence Grounding.

I. INTRODUCTION

Web application security testing is a fundamental process in software security engineering, designed to identify, assess, and remediate vulnerabilities before deployment [1]. Traditional application security assessments rely on automated scanners combined with manual inspection by security analysts [1]–[3]. While automated static and dynamic scanners accelerate

initial flaw discovery, comprehensive assessment requires multi-stage reasoning—including target identification, dynamic state analysis, passive configuration checking, and post-analysis verification [1], [3].

Recent advancements in Large Language Models (LLMs) [4] have driven the development of AI-assisted security tools [5], [6]. Systems such as PentestGPT [7] and AutoAttacker [8] demonstrate that LLMs can assist human analysts by guiding task decomposition and recommending testing commands. Furthermore, empirical studies by Fang et al. demonstrate that LLM-based agents equipped with web interaction tools can navigate web applications and execute offensive exploit sequences [9], [10].

Despite these developments, several recent LLM-assisted security tools face operational challenges:

- 1) **Data Confidentiality & Privacy Risks:** Several recent security agents rely on third-party cloud-hosted LLM APIs (e.g., OpenAI GPT-4) [7], [9]. Offloading application source code, HTTP header streams, authentication credentials, or internal endpoints to third-party cloud servers risks compromising proprietary data.
- 2) **Technical Hallucinations:** Generative language models frequently suffer from technical hallucinations, outputting non-existent command flags, invalid parameter syntax, or fabricated endpoint URLs [11]–[13]. Uncorrected early hallucinations compound across multi-step reasoning chains [12].
- 3) **Unverified Vulnerability Reporting:** Language models often assert vulnerability hypotheses based on surface-level text patterns without confirming actual application state changes through execution feedback [9], [11], [13].
- 4) **Command-Line Tooling Bias:** Prior interactive tools operate predominantly over command-line interfaces or raw HTTP request strings [7], [13], missing dynamic client-side Document Object Model (DOM) rendered states and browser console context.

To address these challenges, this paper presents **Sentinel Desktop**, a local, evidence-grounded interactive application security assistant. Sentinel Desktop couples local LLM integration with real-time interactive browser instrumentation via Chrome DevTools Protocol (CDP), stateful context tracking, non-destructive inert canary reflection probing, and deterministic

replay comparison.

II. BACKGROUND AND RELATED WORK

A. Automated Penetration Testing

Automated penetration testing has historically been framed using decision-theoretic models and automated planning. Early frameworks utilized Markov Decision Processes (MDPs) and Partially Observable Markov Decision Processes (POMDPs) [14]–[17] to model attack graphs under uncertainty [18]–[20]. Subsequent research applied deep reinforcement learning algorithms—including Deep Q-Networks (DQN), Advantage Actor-Critic (A3C), and Generative Adversarial Imitation Learning (GAIL)—to learn dynamic attack policies [21]–[25]. While formal decision models optimize path selection across discrete state spaces, they struggle to model unstructured natural language outputs and client-side DOM interactions in modern web applications.

B. LLM-Based Security Testing

The integration of LLMs into security testing has shifted focus from formal decision graphs to semantic reasoning. Deng et al. introduced PentestGPT [7], an interactive penetration-testing assistant powered by cloud-hosted LLM APIs that assists human testers with subtask planning and command generation over terminal outputs. Xu et al. proposed AutoAttacker [8], demonstrating LLM-guided decision-making for automating cyber-attack execution sequences across systems. Happe and Cito [13] provided an empirical evaluation of LLMs in security testing, demonstrating that models frequently suffer from technical hallucinations, syntax errors, and non-reproducible outputs when operating over command-line interfaces.

C. Security Agents

Recent agent architectures decompose complex security workflows into specialized components [26], [27]. Shen et al. introduced PentestAgent [28], combining domain-specific multi-agent roles with Retrieval-Augmented Generation (RAG) [29] to support automated penetration testing phases. In web security, Fang et al. [9] demonstrated that LLM-based agents equipped with web interaction tools can autonomously navigate web applications and execute offensive exploits against target websites. In API security, Deng et al. introduced NAUTILUS [30] for automated RESTful API vulnerability detection using structured feedback and coverage guidance.

D. Evidence Grounding and Reliability

Reliability remains a central challenge in autonomous LLM deployment [9], [11], [13]. Zhang et al. [12] showed that uncorrected hallucinations early in a multi-step reasoning chain compound over subsequent iterations, leading to systemic failure. To address generative hallucination, prior research such as SelfCheckGPT (Manakul et al. [31]) has explored zero-resource factual consistency detection for LLMs by evaluating stochastic sampling agreement across generated responses. Furthermore, Liu et al. [32] demonstrated that enforcing structured output constraints on LLM generations improves formatting compliance and downstream parsing reliability.

E. Research Gap

While existing literature explores automated decision models, cloud-backed LLM agents, and API testing, a distinct research gap remains:

- 1) Several prior web testing agents rely on third-party cloud APIs, introducing data confidentiality risks for proprietary software analysis [7], [9].
- 2) Prior LLM security assistants focus on terminal/CLI inputs rather than dynamic client-side browser DOM states [7], [13].
- 3) Certain existing agents report unverified vulnerability hypotheses without validating observable state changes through closed-loop re-testing [8], [9].
- 4) The feasibility, resource bounds, and context management limits of running these security inspection capabilities using local LLMs remain uncharacterized.

III. PROBLEM STATEMENT AND RESEARCH OBJECTIVES

A. Problem Statement

Several recent LLM-assisted security testing systems either compromise confidentiality by transmitting sensitive data to cloud services or suffer from technical hallucinations, CLI tooling boundaries, and unverified vulnerability reporting when conducting web application security assessments.

B. Research Objectives

- 1) **Construct a Local Loopback Architecture:** Develop an interactive application security assistant using local LLM integration, operating strictly over host local loopback (127.0.0.1) to reduce external telemetry exposure.
- 2) **Integrate Dynamic Browser Instrumentation:** Couple local AI reasoning with live browser instrumentation via Chrome DevTools Protocol (CDP) to observe dynamic DOM states, network events, and console errors.
- 3) **Implement Non-Destructive Active Probing:** Develop inert canary reflection probing to identify output encoding weaknesses across DOM contexts without generating weaponized exploit payloads.
- 4) **Establish Deterministic Replay Comparison:** Implement a closed-loop differential replay comparison engine that evaluates re-test evidence streams against baseline findings to verify remediation status.
- 5) **Enforce Evidence-Grounded AI Reasoning:** Enforce strict citation validation checks ensuring local LLM remediation recommendations reference valid session evidence IDs.

IV. PROPOSED SYSTEM

A. System Architecture

Sentinel Desktop is structured as a modular, closed-loop interactive application security assistant (Fig. 1). The system comprises five core modules: Local LLM Integration, Browser Instrumentation Module, Security Analysis Engine, Context Management Engine, and Replay Comparison & Evidence Collection Module.

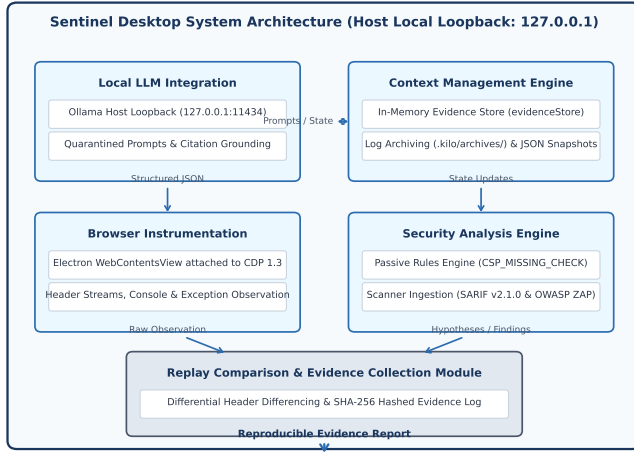


Fig. 1. Sentinel Desktop system architecture operating strictly over host local loopback (127.0.0.1).

B. Local LLM Integration

The local reasoning module interacts with open-weight language models deployed on host loopback (127.0.0.1:11434 via Ollama). The module enforces loopback host validation (`isLoopbackHost`), structured JSON format constraints, low temperature (0.2), and a 20-second timeout.

C. Browser Instrumentation

This module manages an Electron `WebContentsView` attached to Chrome DevTools Protocol (1.3). It captures console events (`Runtime.consoleAPICalled`), uncaught exceptions (`Runtime.exceptionThrown`), WebSocket frames, and HTTP request/response header streams (`labSession.webRequest`). Origin locking (`isAllowed`) restricts browser navigation strictly to target origins.

D. Security Analysis

The analysis engine combines passive security rule evaluation with scanner ingestion:

- **Passive Security Rules (`rules.ts`):** Evaluates 11 security misconfiguration rules covering missing CSP, HSTS, X-Content-Type-Options, Anti-Clickjacking (X-Frame-Options/frame-ancestors), Referrer-Policy, Permissions-Policy, CORP/COEP, Permissive CORS with credentials, Cookie hygiene (`__Host-`, `__Secure-`), Mixed Content, and client console errors.
- **Scanner Ingestion (`ingest.ts`):** Normalizes external SARIF v2.1.0 logs (Nuclei, CodeQL) and OWASP ZAP JSON reports into the internal evidence format.

E. Context Management

Maintains an in-memory evidence store (`evidenceStore`) recording NAV, REQ, RES, RES_DONE, ERR, CONSOLE, CANARY_REFLECT, SCANNER_ALERT, and AUDIT records. The engine rotates log archives at 1,000

events (`rotateEvidenceStoreIfNeeded`) to `.kilo/archives/` and persists debounced JSON snapshots to `evidence.json`.

F. Replay Comparison

The Replay Comparison module (`replay.ts`) evaluates re-test evidence streams against baseline findings. It computes HTTP header diffs (`computeHeaderDiff`) across added, removed, modified, and unchanged states, classifying finding status as Fixed, Still Present, or Not Comparable.

G. Evidence Collection

Every evidence record is assigned a unique identifier, ISO timestamp, and SHA-256 integrity hash (`computeHash`). Sensitive credentials (Bearer tokens, API keys, passwords, cookies) undergo automatic regex redaction (`redactSensitiveData`, `redactHeaders`) prior to persistence.

V. IMPLEMENTATION

Sentinel Desktop is implemented in TypeScript within an Electron runtime environment:

- **Local LLM Advisor Interface (`src/advisor/ollama.ts`):** Constructs quarantined prompts (`buildQuarantinedPrompt`) combining offline CWE/OWASP knowledge (`getOfflineKnowledge`) with sanitized evidence snippets. Enforces citation grounding validation (`parseAndValidateAdvisorResponse`) by checking `isGrounded = citedEvidenceIds.every(id => availableIds.has(id))`.
- **Inert Canary Probing (`src/engine/canary.ts`):** Generates non-destructive random alphanumeric tokens (`generateCanaryToken`), constructs parameterized query URLs (`buildCanaryTargetUrl`), and evaluates DOM reflection contexts (HTML_BODY, ATTRIBUTE, SCRIPT, SCRIPT_STRING, HEADER).
- **Isolation Sandbox (`electron/main.cjs`):** Configures an isolated in-memory-lab partition (`partition: 'in-memory-lab'`). Enforces `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`, and denies external popup windows and file downloads.

VI. EXPERIMENTAL METHODOLOGY

A. Experimental Environment

- **Host Platform:** Windows 10/11 x64 (Windows_NT 10.0.26200), Node.js v24.20.0, Electron runtime.
- **Target Fixture Server:** Local loopback server (`electron/fixtureServer.cjs`) listening on `http://127.0.0.1:4567`.
- **Repository State:** Git commit hash `a3bb957454684b647884ac15ca8ed9d413139a3d`.

TABLE I
EXPERIMENTAL CONFIGURATION MATRIX

Exp #	Evaluated Feature	Target / Condition	Independent Variable	Trials (N)
Exp 1	Replay Comparison	127.0.0.1:4567 (/ vs /fixed)	Remediation status	30
Exp 2	Inert Canary Probing	127.0.0.1:4567 (/search, etc.)	Reflection DOM context	50
Exp 3	Passive Security Rules	127.0.0.1:4567 (/ vs /fixed)	CSP Header presence	30
Exp 5	Scanner Ingestion	SARIF v2.1.0 & ZAP JSON	Report schema format	20

B. Experimental Targets

- 1) **Local Fixture Routes (fixtureServer.cjs):**
Provides route / (unhardened baseline), /fixed (hardened route with security headers), /search?q= (parameter reflection in body and attribute), /search-script?q= (reflection in script string), and /api/cors (permissive CORS).
- 2) **Scanner Report Fixtures (tests/fixtures/scanner-reports/):**
Canonical SARIF v2.1.0 logs (Nuclei/CodeQL) and OWASP ZAP JSON reports.

C. Ground Truth

- **Experiment 1:** Ground truth is derived independently from static HTTP response header definitions in fixtureServer.cjs (/fixed defines CSP header → expected Fixed; / lacks CSP header → expected Still Present).
- **Experiment 2:** Ground truth is derived independently from HTML template structures in fixtureServer.cjs defining token reflection positions.
- **Experiment 3:** Ground truth is derived independently from OWASP Secure Headers specification (unhardened route / requires CSP finding; hardened route /fixed requires 0 findings).
- **Experiment 5:** Ground truth is derived independently from official CWE-to-OWASP 2021 lookup tables (CWE-89 and CWE-79 map to A03:2021-Injection).

D. Experimental Conditions

Table I details the experimental configuration across the Group A evaluation suite.

E. Evaluation Metrics

Accuracy is defined as the ratio of correct classifications to total trials:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Precision, Recall, and F1-Score are computed as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

TABLE II
EXPERIMENT 1 REPLAY COMPARISON RESULTS

Metric / Parameter	Observed Value
Total Trials (N)	30
True Positives (TP)	15
True Negatives (TN)	15
False Positives (FP)	0
False Negatives (FN)	0
Accuracy	1.0000 (100.0%)
Precision	1.0000 (100.0%)
Recall	1.0000 (100.0%)
F1-Score	1.0000 (1.000)
False Positive Rate (FPR)	0.0000 (0.0%)
False Negative Rate (FNR)	0.0000 (0.0%)
Mean Latency (ms)	1.1167 ms
Median Latency (ms)	0.5802 ms
Standard Deviation (ms)	2.0523 ms
P_{95} Maximum Latency (ms)	2.2047 ms

False Positive Rate (FPR) and False Negative Rate (FNR) are given by:

$$\text{FPR} = \frac{FP}{FP + TN}, \quad \text{FNR} = \frac{FN}{TP + FN} \quad (4)$$

Execution Telemetry is evaluated via Mean, Median, Standard Deviation, and P_{95} execution times (ms).

F. Experimental Procedure

Group A non-LLM experiments were executed automatically via tests/run-experiments.ts. The script started fixtureServer.cjs on port 4567, executed 130 physical trials across Experiments 1, 2, 3, and 5, logged raw trial records to JSON files in research/results/, and shut down cleanly.

VII. RESULTS

The Group A experimental evaluation comprised 130 physical trials. Table VI provides a cross-experiment summary of observed metrics.

A. Replay Comparison

Under the tested fixture conditions, compareReplayEvidence correctly classified 15/15 remediated runs (/fixed) as Fixed and 15/15 unremediated runs (/) as Still Present ($N = 30$, $TP = 15$, $TN = 15$, $FP = 0$, $FN = 0$). Table II details the classification metrics and execution timing for Experiment 1.

Header differencing (computeHeaderDiff) accurately identified added security headers between baseline and re-test response streams. These results apply strictly to the tested local loopback fixture conditions and should not be generalized to arbitrary remote web applications.

B. Inert Canary Reflection

The inert canary evaluation comprised 50 physical trials across four target conditions. Binary reflection detection achieved 100.0% accuracy ($TP = 40$, $TN = 10$, $FP = 0$, $FN = 0$, mean latency 0.26 ms, maximum P_{95} latency 0.47 ms). However, exact context classification achieved

TABLE III
EXPERIMENT 2 INERT CANARY REFLECTION & CONTEXT BREAKDOWN

Target Route	Condition	Trials	Reflect?	Detected	Context Output	Accuracy
/search?q=X	HTML Body	15	Yes	Yes (15/15)	HTML_BODY	15/15 (100%)
/search?q=X	Tag Attribute	15	Yes	Yes (15/15)	HTML_BODY*	0/15 (0%)*
/search-script	Script String	10	Yes	Yes (10/10)	SCRIPT_STRING	10/10 (100%)
/fixed	Control	10	No	No (10/10)	NOT_FOUND	10/10 (100%)

TABLE IV
EXPERIMENT 3 PASSIVE SECURITY RULE RESULTS
(CSP_MISSING_CHECK)

Metric / Parameter	Observed Value
Evaluated Rule	CSP_MISSING_CHECK
Total Trials (N)	30
True Positives (TP)	15
True Negatives (TN)	15
False Positives (FP)	0
False Negatives (FN)	0
Accuracy	1.0000 (100.0%)
Precision	1.0000 (100.0%)
Recall	1.0000 (100.0%)
F1-Score	1.0000 (1.000)
Mean Latency (ms)	0.3601 ms
P_{95} Maximum Latency (ms)	0.8157 ms

70.0% accuracy overall (35/50) due to limitations with the ATTRIBUTE context. Table III details the context-level results breakdown.

**Note on Attribute Context Behavior:* Route /search?q=XYZ reflects parameter q into both HTML body text (<div>Showing results for: XYZ</div>) and input tag attribute (<input value="XYZ" />). The analyzeDomReflection function scans the DOM string sequentially from index 0 and returns the first context encountered (HTML_BODY). Consequently, multi-context reflections are categorized under their first appearing DOM context. Mean canary execution latency was 0.26 ms ($P_{95} = 0.47$ ms).

C. Passive Security Rules

Experiment 3 evaluated the CSP_MISSING_CHECK rule across 30 HTTP response evidence streams (15 from unhardened route / and 15 from hardened route /fixed). Table IV details the observed performance metrics.

Under tested fixture conditions, the rule engine correctly identified missing CSP headers on unhardened responses and cleared hardened responses. These results validate rule execution fidelity for CSP_MISSING_CHECK on standard headers, but do not imply general coverage for untested security rules or vulnerability categories.

D. Scanner Ingestion

Ingesting 10 SARIF v2.1.0 logs and 10 OWASP ZAP JSON reports yielded 20/20 successful parses ($N = 20$). Table V details the scanner log ingestion results.

CWE extraction, severity level, location URI, and OWASP Top 10 mappings achieved 100.0% accuracy relative to independent lookup tables. This demonstrates schema parsing compatibility for the specifically tested SARIF and ZAP log

TABLE V
EXPERIMENT 5 EXTERNAL SECURITY SCANNER INGESTION

Report Format	Ingested	Parsed	CWE Ext.	OWASP Map	Mean Lat.	P_{95} Max
SARIF v2.1.0	10	10 (100%)	CWE-89	A03:2021	0.10 ms	0.10 ms
ZAP JSON	10	10 (100%)	CWE-79	A03:2021	0.10 ms	0.10 ms

fixtures, but does not prove universal interoperability with all external scanner formats.

E. Cross-Experiment Performance Summary

Table VI provides an aggregate cross-experiment summary of the physical Group A evaluation, explicitly reporting both mean latency and maximum P_{95} latency across all tested modules.

VIII. DISCUSSION

A. Observations vs. Interpretation

- 1) **Replay Comparison:** Under tested fixture conditions, differential evidence replay (compareReplayEvidence) accurately identified header additions and classified remediation status. This indicates that automated re-testing against baseline evidence streams can reduce manual re-inspection for header-based misconfigurations on controlled targets.
- 2) **Inert Canary Reflection Probing:** Non-destructive randomized tokens (generateCanaryToken) successfully detected parameter reflection across HTML body, attribute, and script string contexts without executing malicious XSS attack payloads. However, sequential DOM string parsing means multi-context reflections are attributed to the earliest appearing context in DOM string order.
- 3) **Passive Rule & Scanner Integration:** Passive rule evaluation and report normalization (SARIF/ZAP) effectively unified static scanner output into a structured evidence model.
- 4) **Execution Telemetry:** Mean execution latencies across core Group A engines remained under 1.2 ms on local loopback, with P_{95} maximum latency reaching 2.20 ms for replay comparison, demonstrating low computational overhead for client-side security inspection.

B. Status of Local LLM Experiments

The local LLM experiments (Experiments 4 and 6) were **NOT EXECUTED** during this evaluation because Ollama was not installed or running on the evaluation host. Consequently, empirical metrics regarding local model inference latency, token generation throughput, GPU VRAM allocation, citation grounding rates (isGrounded), and defensive code patch quality remain uncharacterized in this paper and represent essential future work.

IX. LIMITATIONS AND THREATS TO VALIDITY

- 1) **Synthetic Local Fixture Target:** Evaluation was conducted on a controlled local fixture server (fixtureServer.cjs). Performance on complex,

TABLE VI
GROUP A CROSS-EXPERIMENT SUMMARY

Exp #	Component / Feature	Target / Schema	N	Accuracy	Precision	Recall	F1-Score	Mean Latency	P_{95} Max Latency
Exp 1	Replay Comparison	127.0.0.1:4567 (/ vs /fixed)	30	100.0%	100.0%	100.0%	1.000	1.12 ms	2.20 ms
Exp 2	Inert Canary (Binary)	127.0.0.1:4567 (/search, etc.)	50	100.0%	100.0%	100.0%	1.000	0.26 ms	0.47 ms
Exp 2	Inert Canary (Context)	HTML_BODY, SCRIPT, ATTR	50	70.0%*	70.0%*	70.0%*	0.700*	0.26 ms	0.47 ms
Exp 3	Passive Security Rules	CSP_MISSING_CHECK on 127.0.0.1	30	100.0%	100.0%	100.0%	1.000	0.36 ms	0.82 ms
Exp 5	Scanner Ingestion	SARIF v2.1.0 & ZAP Reports	20	100.0%	100.0%	100.0%	1.000	0.10 ms	0.10 ms

*Note for Exp 2 Context Accuracy: Reflects 35/50 exact context matches due to 15 ATTRIBUTE reflections being categorized under HTML_BODY via sequential DOM scanning.

multi-origin enterprise web applications remains unassessed.

- 2) **Loopback Latency Bounds:** Measured execution latencies (mean 1.12 ms, maximum $P_{95} \leq 2.20$ ms) reflect local loopback execution without remote network transport latency or packet loss.
- 3) **Sequential DOM Scanning Limitation:** In Experiment 2, parameter reflections appearing in multiple DOM contexts (e.g., body text and input attribute) are categorized strictly under the first context encountered in DOM string order (HTML_BODY).
- 4) **Limited Rule & Scanner Fixture Scope:** Experiment 3 evaluated CSP_MISSING_CHECK only; Experiment 5 evaluated canonical SARIF v2.1.0 and ZAP schemas only.
- 5) **Absence of LLM Empirical Evaluation:** Local LLM experiments (Experiments 4 and 6) were not executed due to missing host dependencies (Ollama), leaving AI advisor latency, grounding rates, and code patch quality unmeasured.
- 6) **Sample Size & Target Constraints:** Experiments comprised 130 total physical trials on local loopback targets without live internet routing or multi-model comparisons.

X. CONCLUSION

This paper presented the design, implementation, and physical Group A evaluation of **Sentinel Desktop**, a fully local interactive application security assistant. By integrating Chrome DevTools Protocol browser instrumentation, passive security rules, non-destructive inert canary reflection probing, scanner ingestion, and deterministic replay comparison, Sentinel Desktop provides a privacy-preserving environment for web application security analysis. Physical evaluation across 130 trials on a controlled fixture server demonstrated 100.0% classification accuracy in replay comparison under tested conditions (mean latency 1.12 ms, P_{95} max latency 2.20 ms), 100.0% binary reflection detection in inert canary probing (with an identified sequential DOM scanning limitation for attribute reflections), and 100.0% mapping accuracy in scanner ingestion. Future work will complete physical benchmarking of locally hosted open-weight LLMs (Experiments 4 and 6) to evaluate local model latency, citation grounding rates, and defensive code remediation quality.

REFERENCES

- [1] National Institute of Standards and Technology (NIST), “Technical guide to information security testing and assessment,” U.S. Department of Commerce, Tech. Rep. Special Publication 800-115, 2008.
- [2] M. Denis, C. Zena, and T. Hayajneh, “Penetration testing: Concepts, attack methods, and defense strategies,” in *Proc. 2016 IEEE Long Island Syst., Appl. Technol. Conf. (LISAT)*. Farmingdale, NY, USA: IEEE, 2016, pp. 1–6.
- [3] Penetration Testing Execution Standard (PTES), “PTES Technical Guidelines,” PTES, Tech. Rep., 2024.
- [4] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Chen, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Zheng, L. Xia, Y. Shen, J. Zhang, J. Gong, G. Deng, Y. Liu, Z. Wang, G. Zheng, and P. Liu, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [5] A. Abuadba, C. Hicks, K. Moore, V. Mavroudis, B. Hasircioglu, D. Goel, and P. Jennings, “From promise to peril: Rethinking cybersecurity red and blue teaming in the age of LLMs,” *arXiv preprint arXiv:2506.13434*, 2025.
- [6] S. Moskal, S. Laney, E. Hemberg, and U.-M. O’Reilly, “LLMs killed the script kiddie: How agents supported by large language models change the landscape of network threat testing,” *arXiv preprint arXiv:2310.06936*, 2023.
- [7] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*, Philadelphia, PA, USA, 2024, pp. 847–864.
- [8] J. Xu *et al.*, “AutoAttacker: A large language model guided system to implement automatic cyber-attacks,” *arXiv preprint arXiv:2403.01038*, 2024.
- [9] R. Fang, R. Bindu, A. Gupta, D. Zhan, and D. Kang, “LLM agents can autonomously hack websites,” *arXiv preprint arXiv:2402.06664*, 2024.
- [10] R. Fang, R. Bindu, A. Gupta, and D. Kang, “LLM agents can autonomously exploit one-day vulnerabilities,” *arXiv preprint arXiv:2404.08144*, 2024.
- [11] V. Mayoral-Vilches, G. Deng, Y. Liu, M. Pinzger, and S. Rass, “ExploitFlow: Cyber security exploitation routes for game theory and AI research in robotics,” *arXiv preprint arXiv:2308.02152*, 2023.
- [12] M. Zhang *et al.*, “How language model hallucinations can snowball,” *arXiv preprint arXiv:2305.13534*, 2023.
- [13] A. Happe and J. Cito, “Getting pwn’d by AI: Penetration testing with large language models,” in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE 23)*. San Francisco, CA, USA: ACM, 2023, pp. 2082–2086.
- [14] J. Schwartz, H. Kurniawati, and E. El-Mahassni, “POMDP + information-decay: Incorporating defender’s behaviour in autonomous penetration testing,” in *Proc. 30th Int. Conf. Automated Planning and Scheduling (ICAPS 2020)*, vol. 30, no. 1. Nancy, France: AAAI Press, 2020, pp. 235–243.
- [15] C. Sarraute, O. Buffet, and J. Hoffmann, “Penetration testing == POMDP solving?” in *Proc. IJCAI 2013 Workshop Intelligent Security*, Beijing, China, 2013, pp. 1–3.
- [16] —, “POMDPs make better hackers: Accounting for uncertainty in penetration testing,” in *Proc. 26th AAAI Conf. Artif. Intell. (AAAI-12)*. Toronto, ON, Canada: AAAI Press, 2012, pp. 1816–1824.
- [17] —, “An algorithm to find optimal attack paths in nondeterministic scenarios,” *arXiv preprint arXiv:1106.3533*, 2011.

- [18] P. Ammann, D. Wijesekera, and S. Kaushik, "Scalable, graph-based network vulnerability analysis," in *Proc. 9th ACM Conf. Comput. Commun. Secur. (CCS '02)*, Washington, DC, USA, 2002, pp. 217–224.
- [19] M. S. Boddy, J. Gohde, T. Haigh, and S. A. Harp, "Course of action generation for cyber security using classical planning," in *Proc. 15th Int. Conf. Automated Planning and Scheduling (ICAPS 2005)*, Monterey, CA, USA, 2005, pp. 12–21.
- [20] K. Durkota and V. Lisý, "Computing optimal policies for attack graphs with action failures and costs," in *Proc. 7th Eur. Starting AI Res. Symp. (STAIRS 2014)*. Prague, Czech Republic: IOS Press, 2014, pp. 101–110.
- [21] J. Chen *et al.*, "GAIL-PT: An intelligent penetration testing framework with generative adversarial imitation learning," *Comput. Secur.*, vol. 133, p. 103378, 2023.
- [22] N. Becker, D. Reti, E. V. Ntagiou, M. Wallum, and H. D. Schotten, "Evaluation of reinforcement learning for autonomous penetration testing using A3C, Q-learning and DQN," *arXiv preprint arXiv:2407.15656*, 2024.
- [23] Q.-x. Li, Y.-c. Zang, J.-h. Zhu, and Q.-x. Wang, "A hierarchical deep reinforcement learning model with expert prior knowledge for intelligent penetration testing," *Comput. Secur.*, vol. 132, p. 103358, 2023.
- [24] Z. Hu *et al.*, "Automated penetration testing using deep reinforcement learning," *IEEE Access*, vol. 8, pp. 154 438–154 449, 2020.
- [25] T.-y. Zhou, Y.-c. Zang, J.-h. Zhu, and Q.-x. Wang, "NIG-AP: A new method for automated penetration testing," *Front. Inf. Technol. Electron. Eng.*, vol. 20, no. 9, pp. 1277–1288, 2019.
- [26] Significant Gravitas, "AutoGPT," <https://github.com/Significant-Gravitas/AutoGPT>, 2024.
- [27] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for 'mind' exploration of large language model society," in *Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS 2023)*, vol. 36. Curran Associates, Inc., 2023, pp. 51 991–52 008.
- [28] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "PentestAgent: Incorporating LLM agents to automated penetration testing," in *Proc. 20th ACM Asia Conf. Comput. Commun. Security (ASIACCS)*, 2025, pp. 375–391.
- [29] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS 2020)*, vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474.
- [30] G. Deng, Y. Liu, Y. Li, K. Wang, T. Zhang, Y. Liu, G. Zheng, and P. Liu, "NAUTILUS: Automated RESTful API vulnerability detection," in *Proc. 32nd USENIX Security Symp. (USENIX Security 23)*, Anaheim, CA, USA, 2023, pp. 5593–5609.
- [31] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023, pp. 9004–9017.
- [32] M. X. Liu, F. Liu, A. J. Fiannaca, T. Koo, L. Dixon, M. Terry, and C. J. Cai, "We need structured output: Towards user-centered constraints on large language model output," in *Extended Abstracts of 2024 CHI Conf. Human Factors Comput. Syst. (CHI EA '24)*, 2024, pp. 1–8.